Distributed Systems

Monsoon 2025

Lecture 5

International Institute of Information Technology

Hyderabad, India

Some slides borrowed from the MPI tutorial at Argonne National Laboratory

Distributed Programming

- Understand the difficulty of writing distributed programs.
- Know some distributed programming models/ frameworks developed over the years.
  - Practice on some of these models to be issued soon as Homework 2

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.
  - No shared memory
    - Use messages for communication along with their semantics.

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.
  - No shared memory
    - Use messages for communication along with their semantics.
  - Geographical separation
    - Can allow for wider scope of operations, e.g., SETI

Distributed Programming

- Recall that distributed systems are characterized as a collection of autonomous computers communicating over a network of links.
- Some typical features of such a system are:
  - No common physical clock
    - Clocks can drift and no notion of a common clock.
    - Systems can be asynchronous too.
  - No shared memory
    - Use messages for communication along with their semantics.
  - Geographical separation
    - Can allow for wider scope of operations, e.g., SETI
  - Autonomy
    - Processors are loosely coupled but cooperate with each other
  - Heterogeneity
    - All processors need not be alike.

Challenges of Distributed Programming

- Distributed programming is inherently challenging.
- Need to ensure that several geographically separate computers collaborate
  - efficiently,
  - reliably,
  - transparently, and
  - scalable manner
- One can also talk of inherent complexity and accidental complexity

Challenges of Distributed Programming

- One can also talk of inherent complexity and accidental complexity [Check D. Schmidt, VU]
- Inherent complexity due to
  - Latency: Computers can be far apart
  - Reliability: Recovering from failures of individual or collection of computers
  - Partitioning: Sometimes, failures can partition the system.
  - Ordering: Lack of global clock means message/event ordering is difficult
  - Security: Program as well as computational aspects

Challenges of Distributed Programming

- Distributed programming is inherently challenging.
- Need to ensure that several geographically separate computers collaborate
  - Efficiently, reliably, transparently, and scalable manner
- One can also talk of inherent complexity and accidental complexity
- Inherent complexity due to
  - Latency, Reliability, Partitioning, Ordering, Security
- Accidental complexity due to
  - Low-level APIs: Not enough abstraction
  - Poor debugging tools: Poor fault location
  - Algorithmic decomposition: Choice of algorithmic techniques
  - Continuous re-invention: Key components can change!

Two Frameworks for Distributed Programming

- We will spend rest of this module on introducing frameworks for programming distributed systems.
  - MPI
  - Map-Reduce
  - gRPC

What is MPI?

- A *message-passing library specification*
  - not a language or compiler specification
  - not a specific implementation or product
- For parallel computers, clusters, and heterogeneous networks
- Full-featured
  - Designed to provide access to advanced parallel hardware for end users library writers and tool developers

MPI Sources

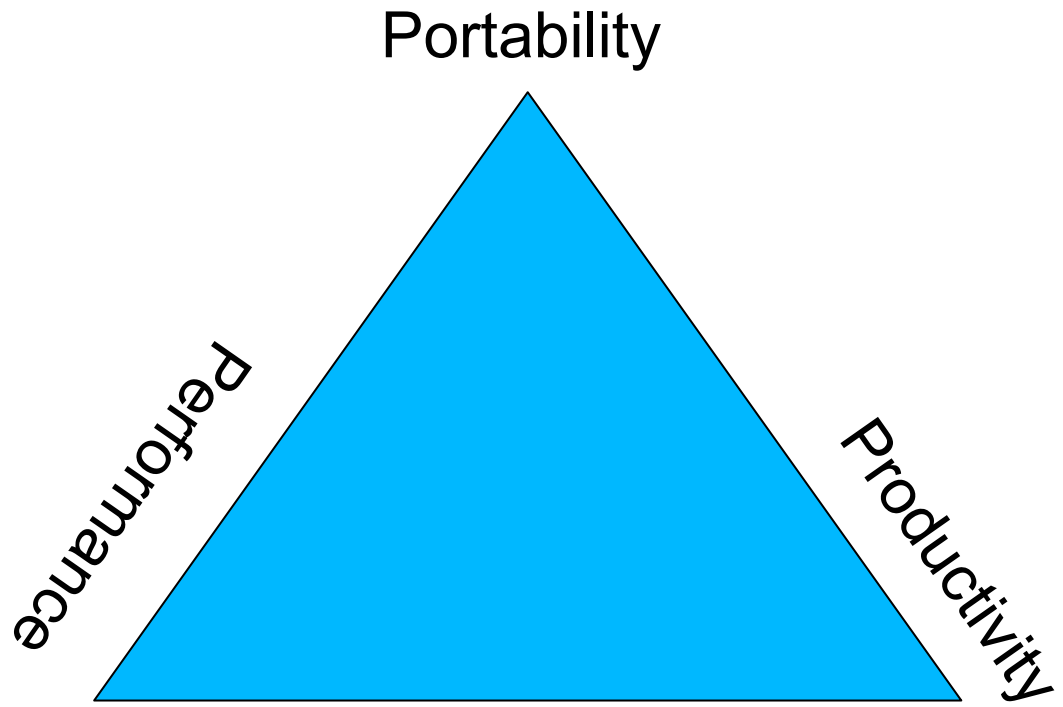
- The Standard itself: at http://www.mpi-forum.org
  - All MPI official releases, in both postscript and HTML

Books:

- *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, by Gropp, Lusk, and Skjellum, MIT Press, 1994.
- *MPI: The Complete Reference,* by Snir, Otto, Huss-Lederman, Walker, and Dongarra, MIT Press, 1996.
- *Designing and Building Parallel Programs*, by Ian Foster, Addison-Wesley, 1995.
- *Parallel Programming with MPI*, by Peter Pacheco, Morgan-Kaufmann, 1997.
- *MPI: The Complete Reference Vol 1 and 2,*MIT Press, 1998(Fall).

Other information on Web: http://www.mcs.anl.gov/mpi

Why Use MPI?



- MPI provides a powerful, efficient, and *portable* way to express parallel programs
- MPI was explicitly designed to enable libraries…
- … which may eliminate the need for many users to learn (much of) MPI

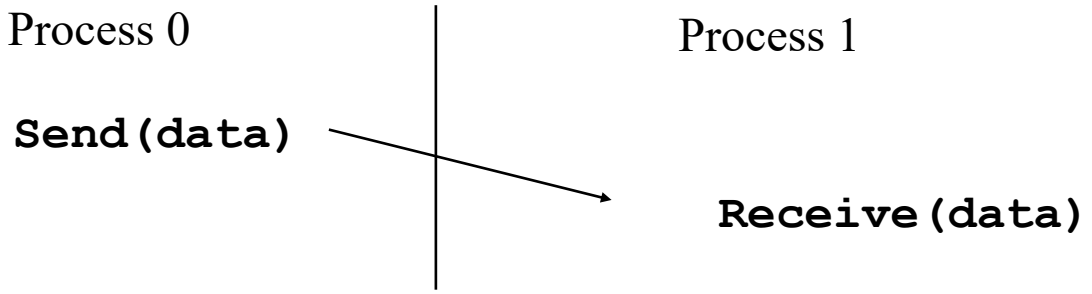
A Minimal MPI Program (C)

```
#include "mpi.h"
#include <stdio.h>

int main( int argc, char *argv[] )
{
    MPI_Init( &argc, &argv );
    printf( "Welcome to the MPI World! \n" );
    MPI_Finalize();
    return 0;
}
```

The Message-Passing Model

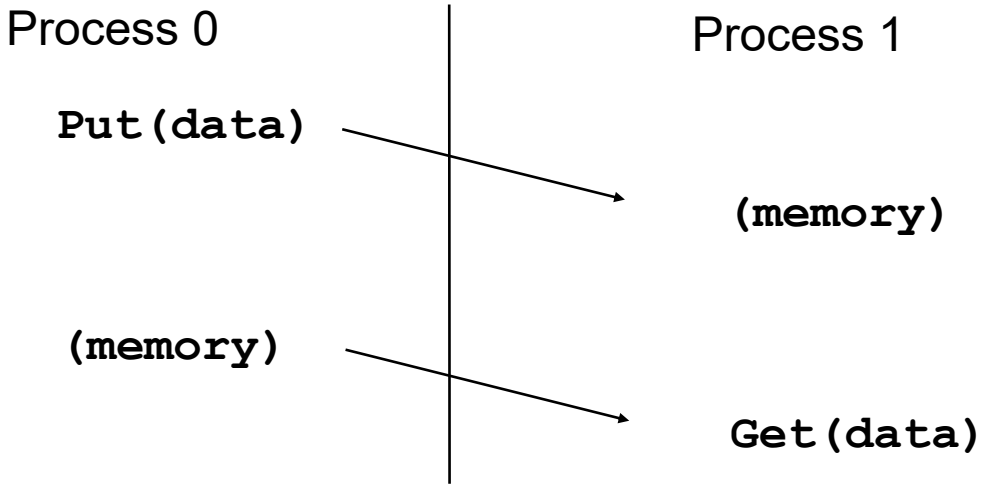
- A *process* is (traditionally) a program counter and address space.
- Processes may have multiple *threads* (program counters and associated stacks) sharing a single address space.
- MPI is for communication among processes, which have separate address spaces.
- Interprocess communication consists of
  - Synchronization
  - Movement of data from one process's address space to another's.

Cooperative Operations for Communication



- The message-passing approach makes the exchange of data *cooperative*.
- Data is explicitly *sent* by one process and *received* by another.
- An advantage is that any change in the receiving process's memory is made with the receiver's explicit participation.
- Communication and synchronization are combined.

One-Sided Operations for Communication



- One-sided operations between processes include remote memory reads and writes
- Only one process needs to explicitly participate.
- An advantage is that communication and synchronization are decoupled
- One-sided operations are part of MPI-2.

Finding Out About the Environment

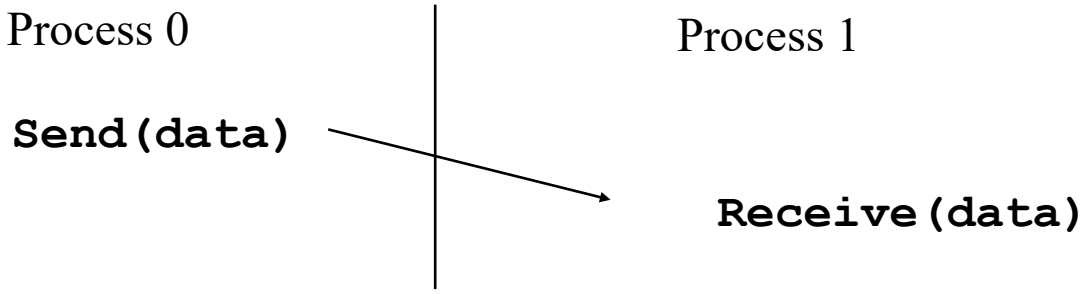
- Two important questions that arise early in a parallel program are:
- How many processes are participating in this computation?
- Which one am I?
- MPI provides functions to answer these questions:
  - **`MPI_Comm_size`** reports the number of processes.
  - **`MPI_Comm_rank`** reports the *rank*, a number between 0 and size-1, identifying the calling process

Better Hello (C)

```
#include "mpi.h"
#include <stdio.h>

int main( int argc, char *argv[] )
{
    int rank, size;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    printf( "I am %d of %d\n", rank, size );
    MPI_Finalize();
    return 0;
}
```

MPI Basic Send/Receive

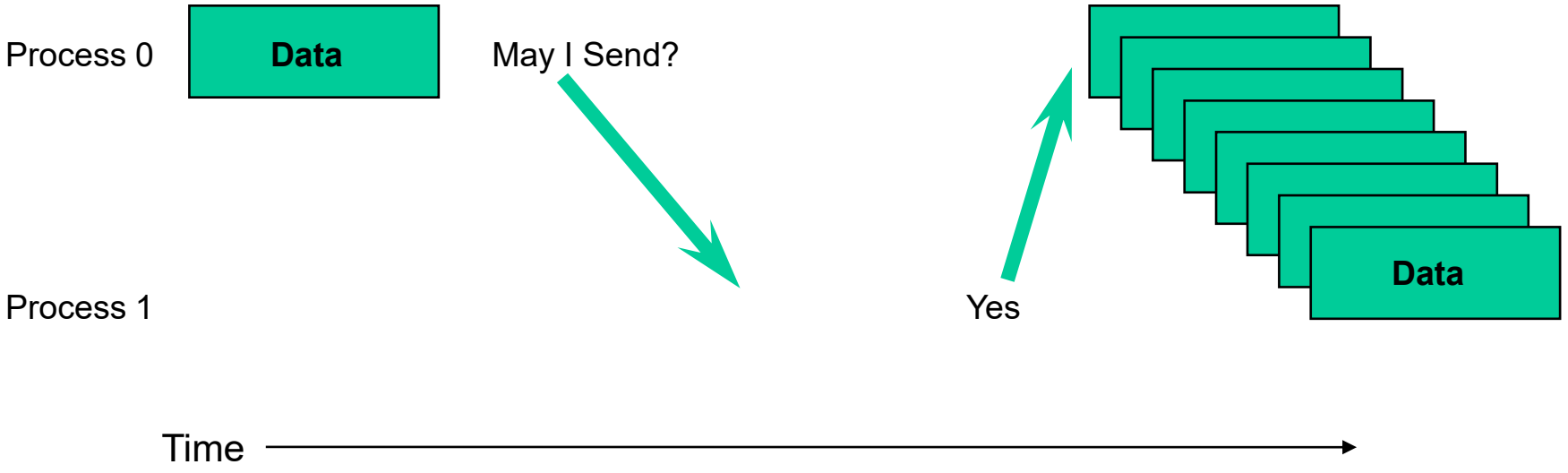
- We need to fill in the details in



- Things that need specifying:
  - How will “data” be described?
  - How will processes be identified?
  - How will the receiver recognize/screen messages?
  - What will it mean for these operations to complete?

What is message passing?

Data transfer plus synchronization



- Requires cooperation of sender and receiver
- Cooperation not always apparent in code

Some Basic Concepts

- Processes can be organized into *groups*.
- Each message is sent in a *context* and must be received in the same context.
- A group and context together form a *communicator*.
- A process is identified by its *rank* in the group associated with a communicator.
- There is a default communicator whose group contains all initial processes, called **`MPI_COMM_WORLD`**.

MPI Basic (Blocking) Send

MPI_SEND (start, count, datatype, dest, tag, comm)

- The message buffer is described by (**`start, count, datatype`**).
- The target process is specified by **`dest`**, which is the rank of the target process in the communicator specified by **`comm`**.
- When this function returns, the data has been delivered to the system, and the buffer can be reused.
- The message may not have been received by the target process.

MPI Basic (Blocking) Receive

- MPI_RECV(start, count, datatype, source, tag, comm, status)

- Waits until a matching (on **`source`** and **`tag`**) message is received from the system, and the buffer can be used.
  - **`source`** is rank in communicator specified by **`comm`**, or **`MPI_ANY_SOURCE`**.
  - **`status`** contains further information
- Receiving fewer than **`count`** occurrences of **`datatype`** is OK, but receiving more is an error.

Retrieving Further Information

- **`Status`** is a data structure allocated in the user's program.

In C:

```
int recvd_tag, recvd_from, recvd_count;
MPI_Status status;
MPI_Recv(..., MPI_ANY_SOURCE, MPI_ANY_TAG, ..., &status )
recvd_tag  = status.MPI_TAG;
recvd_from = status.MPI_SOURCE;
MPI_Get_count( &status, datatype, &recvd_count );
```

In Fortran:

```
integer recvd_tag, recvd_from, recvd_count
integer status(MPI_STATUS_SIZE)
call MPI_RECV(..., MPI_ANY_SOURCE, MPI_ANY_TAG, .. status, ierr)
tag_recvd  = status(MPI_TAG)
recvd_from = status(MPI_SOURCE)
call MPI_GET_COUNT(status, datatype, recvd_count, ierr)
```

Why Datatypes?

- Since all data is labeled by type, an MPI implementation can support communication between processes on machines with very different memory representations and lengths of elementary datatypes (heterogeneous communication).
- Specifying application-oriented layout of data in memory
  - Reduces memory-to-memory copies in the implementation
  - allows the use of special hardware (scatter/gather) when available

Tags and Contexts

- Separation of messages used to be accomplished by use of tags, but
  - this requires libraries to be aware of tags used by other libraries.
  - this can be defeated by use of “wild card” tags.
- Contexts are different from tags
  - no wild cards allowed
  - allocated dynamically by the system when a library sets up a communicator for its own use.
- User-defined tags still provided in MPI for user convenience in organizing application
- Use MPI_Comm_split to create new communicators

MPI in a Simple Way

Many parallel programs can be written using just these six functions, only two of which are non-trivial:

- `MPI_INIT`
- `MPI_FINALIZE`
- `MPI_COMM_SIZE`
- `MPI_COMM_RANK`
- `MPI_SEND`
- `MPI_RECV`

Point-to-point (send/recv) isn't the only way...

Introduction to Collective Operations in MPI

Collective operations are called by all processes in a communicator.

- **MPI_BCAST** distributes data from one process (the root) to all others in a communicator.
  - MPI_BCAST(void *buffer, count, MPI_Datatype, source, Communicator_group_)
- **MPI_REDUCE** combines data from all processes in communicator and returns it to one process.
    - MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype, MPI_Op op, int root, MPI_Comm comm)
- In many numerical algorithms, **SEND/RECEIVE** can be replaced by **BCAST/REDUCE**, improving both simplicity and efficiency.

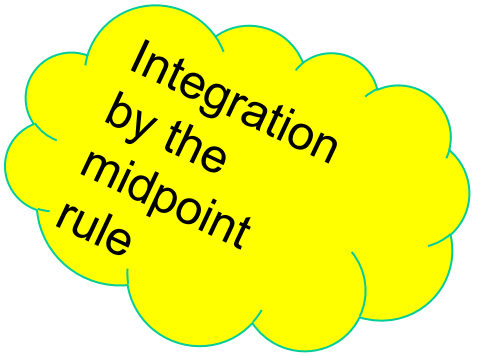
Example: PI in C -1

- Use the integration fact that $\int_0^1 \frac{1}{1+x^2} dx = \frac{\pi}{4}$.

```
#include "mpi.h"
#include <math.h>
int main(int argc, char *argv[])
{
   int done = 0, n, myid, numprocs, i, rc;
   double PI25DT = 3.141592653589793238462643;
   double mypi, pi, h, sum, x, a;
   MPI_Init(&argc,&argv);
   MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
   MPI_Comm_rank(MPI_COMM_WORLD,&myid);
   while (!done)  {
     if (myid == 0) {
       printf("Enter the number of intervals: (0 quits) ");
       scanf("%d",&n);
     }
     MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
     if (n == 0) break;
```

Example: PI in C - 2

```
    h   = 1.0 / (double) n;
    sum = 0.0;
    for (i = myid + 1; i <= n; i += numprocs) {
      x = h * ((double)i - 0.5);
      sum += 4.0 / (1.0 + x*x);
    }
    mypi = h * sum;
    MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0,
               MPI_COMM_WORLD);
    if (myid == 0)
      printf("pi is approximately %.16f, Error is %.16f\n",
             pi, fabs(pi - PI25DT));
  }
  MPI_Finalize();
  return 0;
}
```



The Program Explained in Detail

1. The while loop repeats the program forever until the number of intervals is set as 0.
The "break" statement is the only way to exit the program.

2. The start and end points of an interval are of the kind [i/n, (i+1)/n] as the [0,1] interval is split into n sub-intervals. The index i also jumps by numprocs each iteration. The midpoint of the interval is x := (i-0.5)*1/n.

3. The function value at the mid point is f(x) = 4/(1+x^2).

4. The mid point rule indicates that the area of each interval is approximated by the length of the interval multiplied by the function value at the mid point of the interval.

5. Since each interval is exactly h = 1/n long, h is multiplied by the sum of the function values across the iterations.

Alternative 6 Functions for Simplified MPI

- **MPI_INIT**
- **MPI_FINALIZE**
- **MPI_COMM_SIZE**
- **MPI_COMM_RANK**
- **MPI_BCAST**
- **MPI_REDUCE**

What else is needed (and why)?

Sources of Deadlocks

- Send a large message from process 0 to process 1
  - If there is insufficient storage at the destination, the send must wait for the user to provide the memory space (through a receive)
- What happens with

Process 0	Process 1
`Send(1)`	`Send(0)`
`Recv(1)`	`Recv(0)`

- This is called “unsafe” because it depends on the availability of system buffers

Some Solutions to the “unsafe” Problem

Order the operations more carefully:

Process 0	Process 1
`Send(1)`	`Recv(0)`
`Recv(1)`	`Send(0)`

- Use non-blocking operations:

Process 0	Process 1
`Isend(1)`	`Isend(0)`
`Irecv(1)`	`Irecv(0)`
`Waitall`	`Waitall`

When to use MPI

- Portability and Performance
- Irregular Data Structures
- Building Tools for Others Libraries
- Need to Manage memory on a per processor basis

When *not* to use MPI

- Solution (e.g., library) already exists
  - Check http://www.mcs.anl.gov/mpi/libraries.html
- Require Fault Tolerance

Summary

- The parallel computing community has cooperated on the development of a standard for message-passing libraries.
- There are many implementations, on nearly all platforms.
- MPI subsets are easy to learn and use.
- Lots of MPI material is available.

MPI Sources

- The Standard itself: at http://www.mpi-forum.org
  - All MPI official releases, in both postscript and HTML

Books:

- *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, by Gropp, Lusk, and Skjellum, MIT Press, 1994.
- *MPI: The Complete Reference,* by Snir, Otto, Huss-Lederman, Walker, and Dongarra, MIT Press, 1996.
- *Designing and Building Parallel Programs*, by Ian Foster, Addison-Wesley, 1995.
- *Parallel Programming with MPI*, by Peter Pacheco, Morgan-Kaufmann, 1997.
- *MPI: The Complete Reference Vol 1 and 2,*MIT Press, 1998(Fall).

Other information on Web: http://www.mcs.anl.gov/mpi

Distributed Computing Platforms

- We will study yet another distributed computing platform that has gained recent popularity.
- We will study what this platform and its recent avatars offer.

Distributed Computing

- Distributed Computing involves
  - Clusters of machines connected over network
  - Distributed Storage
    - Disks attached to clusters of machines
    - Network Attached Storage

- **Commodity** clusters
  - Commodity: Available off the shelf at large volumes
  - Lower Cost of Acquisition
  - Cost vs. Performance
  - Low disk bandwidth, and high network latency
  - CPUs typically comparable

Distributed Computing

- Distributed Computing involves
  - Clusters of machines connected over network
  - Distributed Storage
    - Disks attached to clusters of machines
    - Network Attached Storage

How can we make effective use of multiple machines?

- **Commodity** clusters
  - Commodity: Available off the shelf at large volumes
  - Lower Cost of Acquisition
  - Cost vs. Performance
  - Low disk bandwidth, and high network latency
  - CPUs typically comparable

How can we use many of such machines of modest capability?

However,

- Commodity clusters have lower reliability
  - Mass-produced
  - Cheaper materials
  - Smaller lifetime (~3 years)
- *How can applications easily deal with failures?*
- *How can we ensure availability in the presence of faults?*
- *While at the same time…*

While at the same time,

- Realize that programming distributed systems is difficult
  - Divide a job into multiple tasks
  - Understand dependencies between tasks: Control, Data
  - Coordinate and synchronize execution of tasks
  - Pass information between tasks
  - Avoid race conditions, deadlocks
- Parallel and distributed programming models/ languages/ abstractions/platforms try to make these easy
  - E.g. Assembly programming vs. C++ programming
  - E.g. C++ programming vs. Matlab programming

Enter Map-Reduce

- **MapReduce** is a distributed data-parallel programming model from Google
  - Introduced close to 20 years ago.
- MapReduce works best with a distributed file system, called **Google File System (GFS)**
- **Hadoop** is the open source framework implementation from Apache that can execute the MapReduce programming model
- **Hadoop Distributed File System (HDFS)** is the open source implementation of the GFS design
- Amazon's PaaS is yet another implementation of Map-Reduce

Map-Reduce

"A simple and powerful interface that enables automatic parallelization and distribution of large-scale computations, combined with an implementation of this interface that achieves high performance on large clusters of commodity PCs."

Dean and Ghermawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI, 2004

A High Level View

- Map Reduce provides
  - Clean abstraction for programmers
  - Automatic parallelization & distribution
  - Fault-tolerance
  - A batch data processing system
  - Status and monitoring tools

Map-Reduce Vs. RDBMS

- Why not RDBMS – the old data workhorse?
  - RDBMS suffer from a huge seek time resulting in huge access times.
- Map-Reduce comes with the implicit assumption that nearly the entire dataset is relevant for each query.
  - MapReduce is therefore akin to a batch query processor.
- MapReduce is a good fit for problems that need to analyze the whole dataset, in a batch fashion, particularly for ad hoc analysis.

Map-Reduce Vs. RDBMS

- An RDBMS is good for point queries or updates, where the dataset has been indexed to deliver low-latency retrieval and update times of a relatively small amount of data.
- MapReduce suits applications where the data is written once, and read many times, whereas a relational database is good for datasets that are continually updated.
- MapReduce works well on unstructured or semistructured data

Map-Reduce Vs. RDBMS

- Relational data is often normalized to retain its integrity and remove redundancy.
- Normalization poses problems for MapReduce,
  - reading a record a nonlocal operation,
  - one of the central assumptions that MapReduce makes is that it is possible to perform (high-speed) streaming reads and writes.
- A web server log is a good example of a set of records that is not normalized
  - the client hostnames are specified in full each time, even though the same client may appear many times
  - Hence, logfiles are particularly well-suited to analysis with MapReduce

Map-Reduce Vs. RDBMS

- Over time, however, the differences between relational databases and MapReduce systems are reducing.
- Relational databases starting to incorporate some of the ideas from MapReduce
  - Aster Data's and Greenplum's databases
- Higher-level query languages built on MapReduce (such as Pig and Hive) make MapReduce systems more approachable to traditional database programmers.

Map-Reduce

- MapReduce might sound like quite a restrictive programming model
- Mappers and reducers run with very limited coordination between one another.
- Typical problems that we can use Map-Reduce programming model are from image analysis, to graph-based problems, to machine learning algorithms.
- It can’t solve every problem, of course, but it is a general data-processing tool

MapReduce: Data-parallel Programming Model

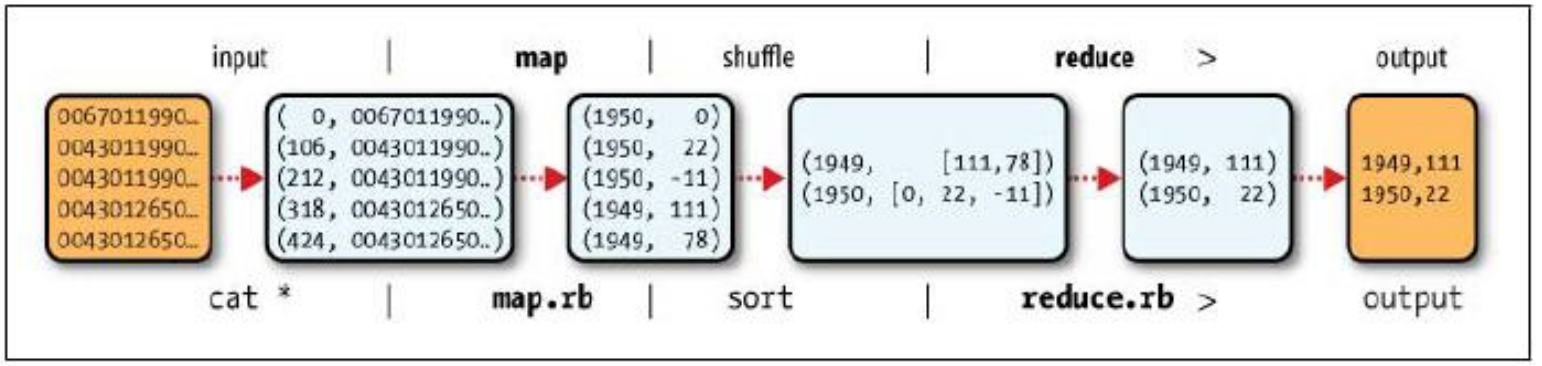
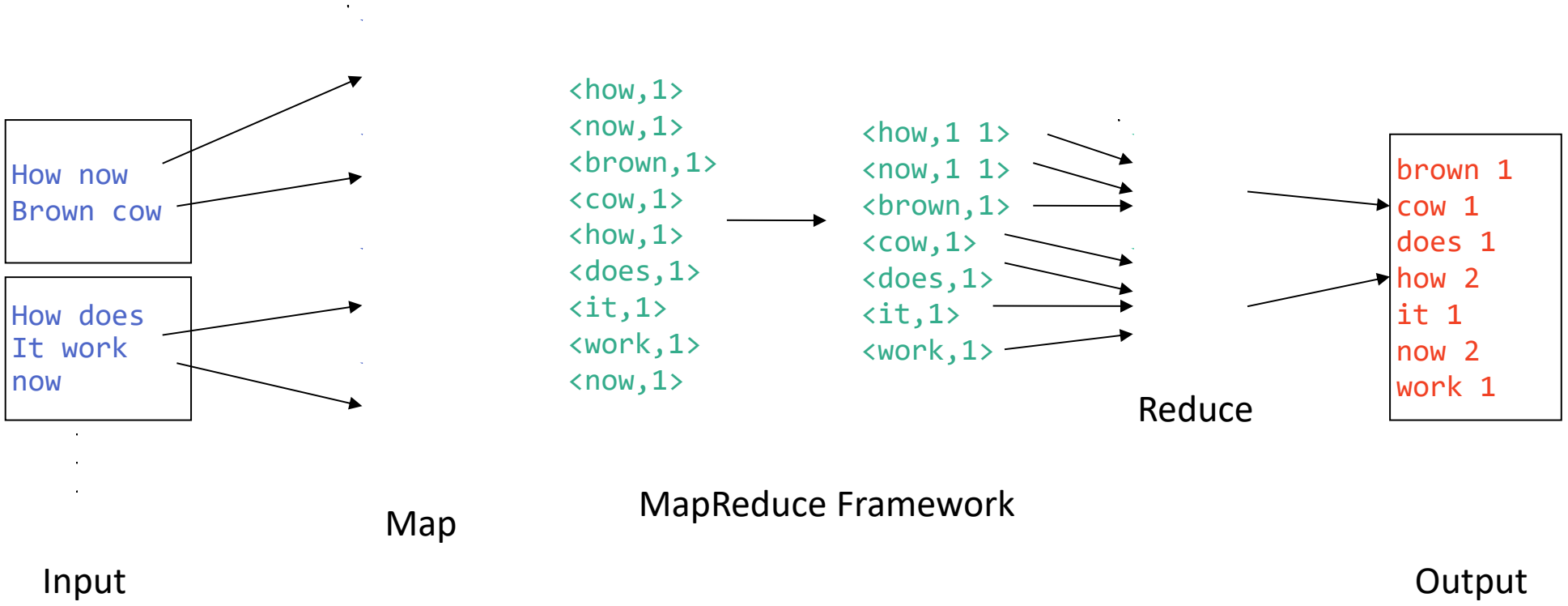


Figure 2-1. MapReduce logical data flow

- Process data using map & reduce functions
- **map(k_i, v_i) → List<k_m, v_m>[]**
  - ***map*** is called on every input item
  - Emits a series of intermediate key/value pairs
- All values with a given key are **grouped** together
- **reduce(k_m, List<v_m>[]) → List<k_r, v_r>[]**
  - ***reduce*** is called on every unique key & all its values
  - Emits a value that is added to the output

MapReduce: Word Count

Map(k1,v1) → list(k2,v2)
Reduce(k2, list(v2)) → list(k2,v2)



Distributed Word Count

Map

- Input records from the data source
  - lines out of files, rows of a database, etc.
- Passed to map function as key-value pairs
  - Line number, line value
- map() produces *zero or more intermediate values*, each associated with an output key.

Map

- **Example: Upper-case Mapper**

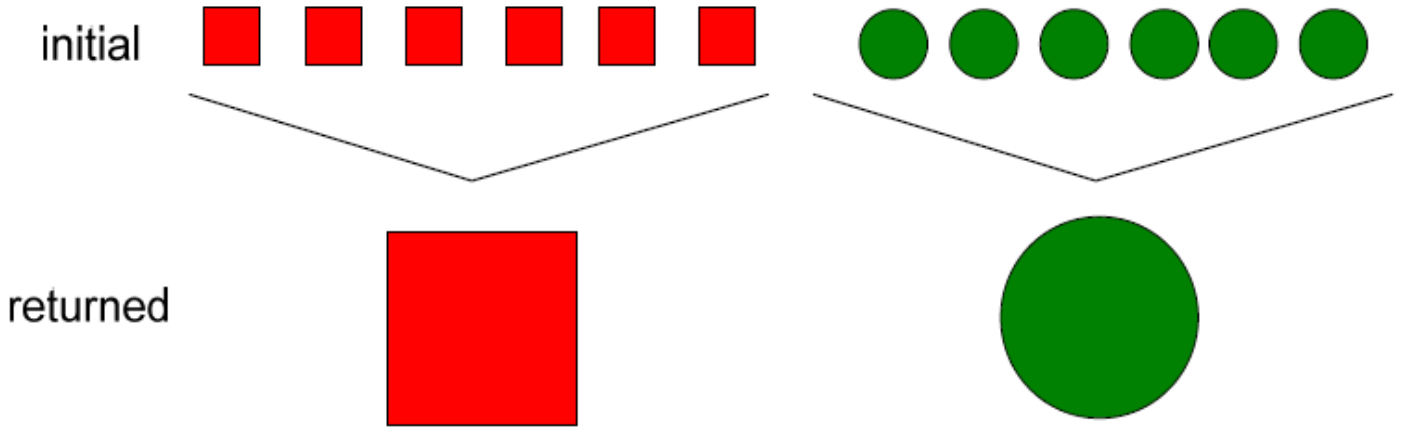
```
map(k, v) { emit(k.toUpper(), v.toUpper()); }

("foo", "bar") → ("FOO", "BAR")
("Foo", "other") → ("FOO", "OTHER")
("key2", "data") → ("KEY2", "DATA")
```

- **Example: Filter Mapper**

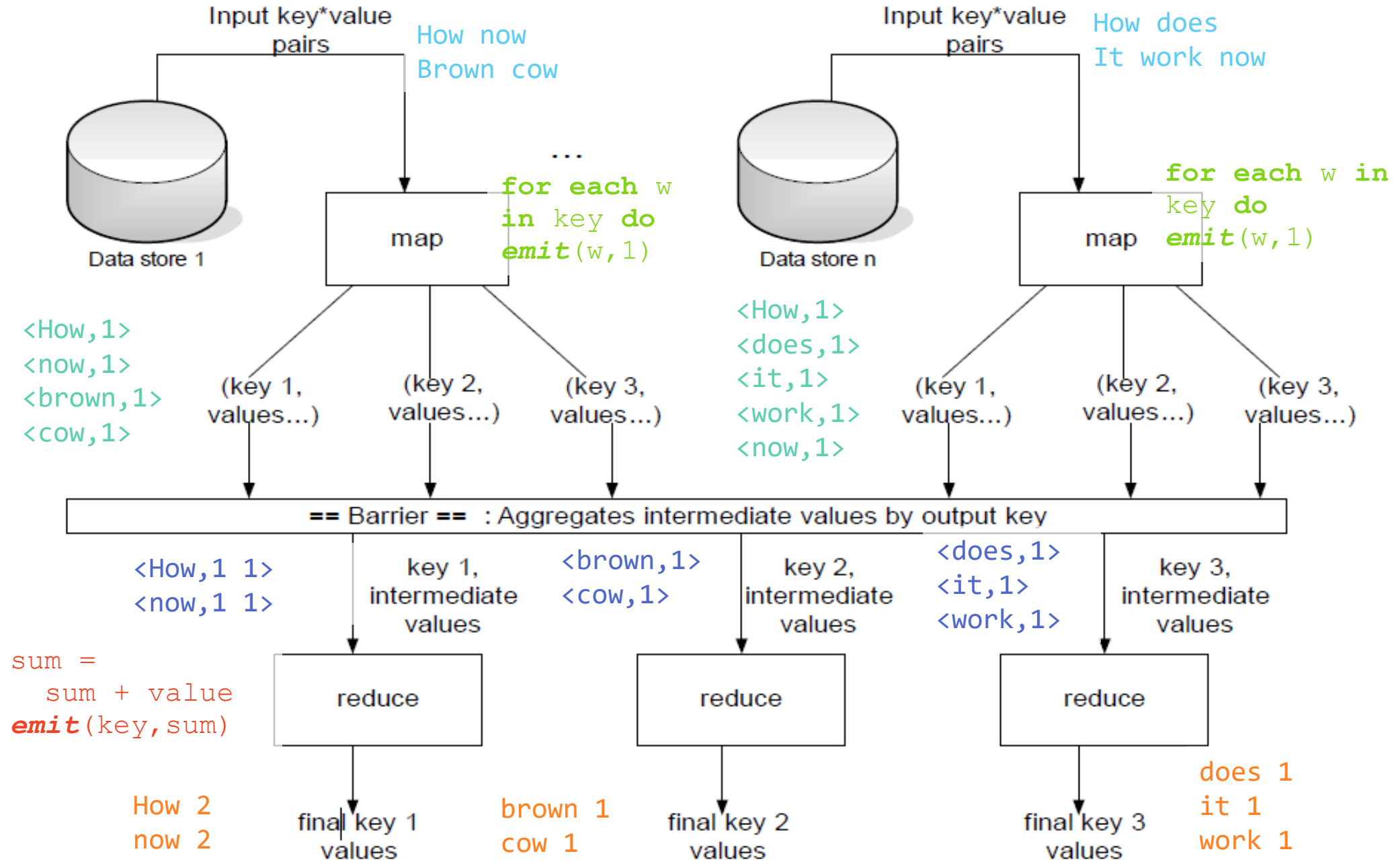
```
map(k, v) { if (isPrime(v)) then emit(k, v); }

("foo", 7) → ("foo", 7)
("test", 10) → () //nothing emitted
```

Reduce

- All the intermediate values from map for a given output key are combined together into a list
- reduce() combines these intermediate values into one or more final values for that same output key … Usually one final value per key



MapReduce: Word Count Drilldown



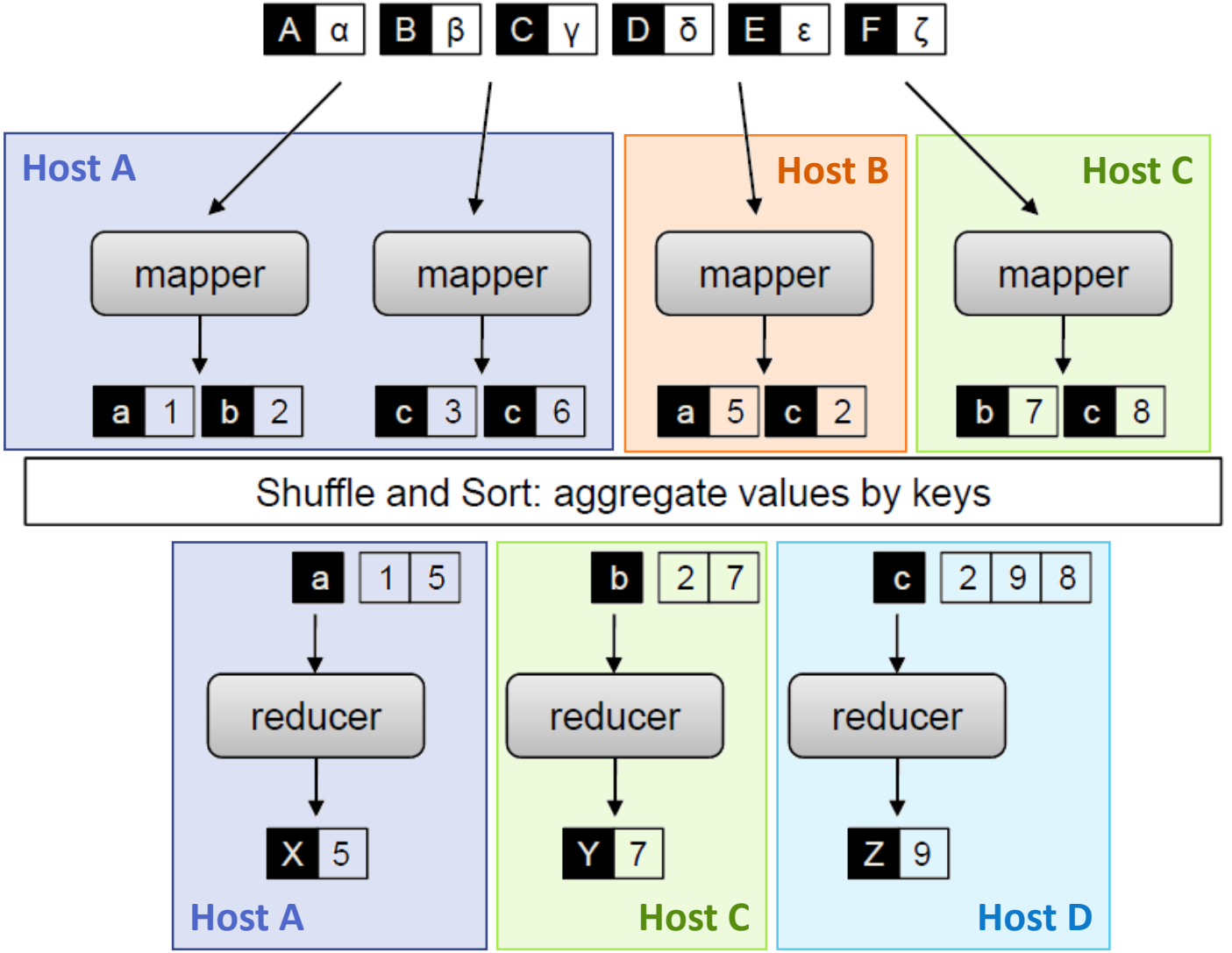
Mapper/Reducer Tasks *vs.* Map/Reduce Methods

- Number of Mapper and Reducer tasks is specified by user
- Each **Mapper/Reducer task** can make multiple calls to **Map/Reduce method**, sequentially
- Mapper and Reducer tasks may run on different machines
- Implementation framework decides
  - Placement of Mapper and Reducer tasks on machines
  - Keys assigned to mapper and reducer tasks
  - But can be controlled by user…

Shuffle & Sort: *The Magic happens here!*

- **Shuffle** does a "group by" of keys from all mappers
  - Similar to SQL groupBy operation
- **Sort** of *local keys* to *Reducer task* performed
  - Keys arriving at each reducer are sorted
  - No sorting guarantee of keys across reducer tasks
- No ordering guarantees of values for a key
  - Implementation dependent
- Shuffle and Sort *implemented efficiently* by framework

Map-*Shuffle-Sort*-Reduce



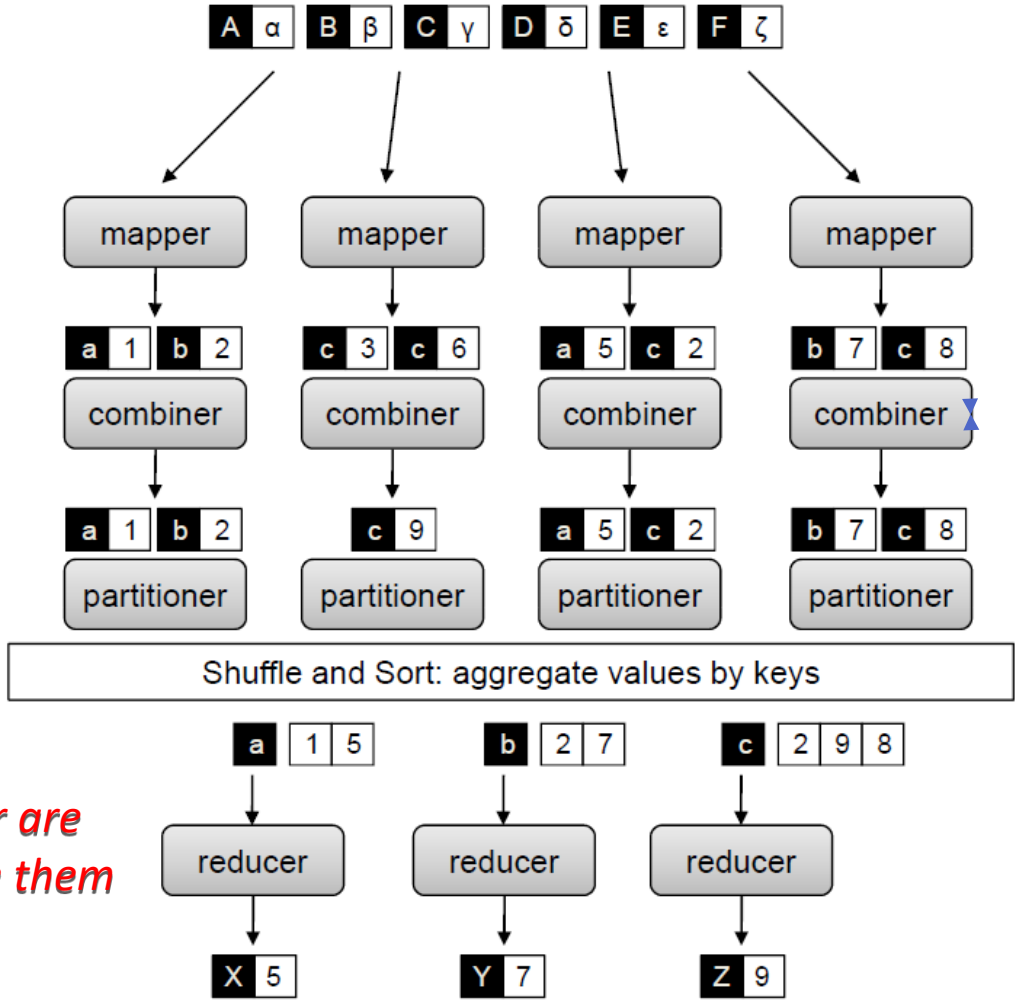
Optimization: **Combiner**

- Logic runs on output of Map tasks, on the map machines
  - "Mini-Reduce," only on local Map output
- Output of Combiner sent to shuffle
  - Saves bandwidth before sending data to Reducers
- Same *input* and *output types* as Map's *output* type
  - `Map(k,v) → (k',v')`
  - `Combine(k',v'[]) → (k',v')`
  - `Reduce(k',v'[]) → (k'',v'')`

Optimization: **Partitioner**

- Decides assignment of intermediate keys grouped to specific Reducer tasks
  - Affects the load on each reducer task
- Sorting of local keys for Reducer task done after partitioning
- Default is hash partitioning
  - **`HashPartitioner(key, nParts)` → `part`**
  - Number of Reducer (**nParts**) tasks known in advance
  - Returns a partition number [0, nParts)
  - Default partitioner balances number of keys per Reducer … *assuming uniform key distribution*
  - May not balance the number of values processed by a Reducer

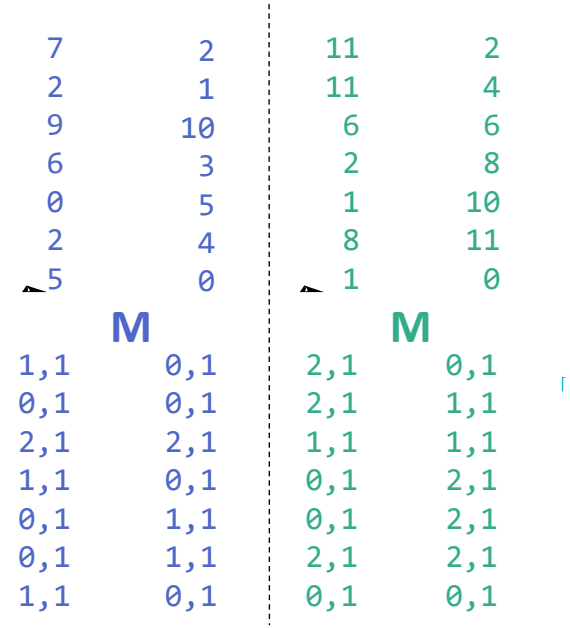
Map-*MiniShuffle-Combine-Partition*-Shuffle-Sort-**Reduce**

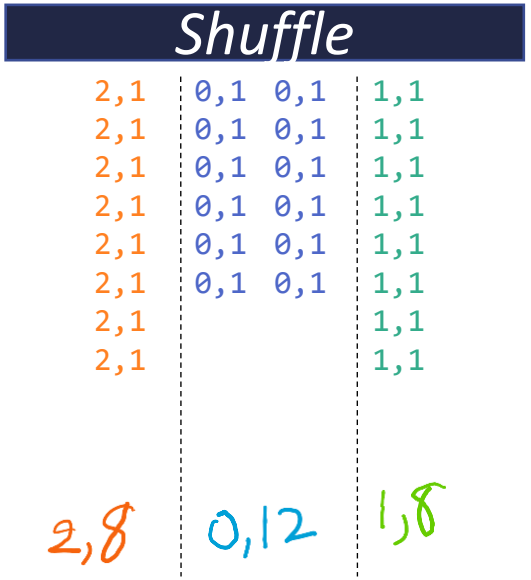


Combine & Partition phases could be interchanged, based on implementation

Combiner & Partitioner are powerful constructs. Use them wisely!

MapReduce for Histogram





Data transfer & shuffle between Map & Reduce **(28 items)**

```
int bucketWidth = 4 // input

Map(k, v) {
emit(floor(v/bucketWidth), 1)
// <bucketID, 1>
}

// one reduce per bucketID
Reduce(k, v[]){
sum=0;
foreach(n in v[])  sum++;
emit(k, sum)
// <bucketID, frequency>
}
```

Combiner Advantage

- Mini-Shuffle lowers the overall cost for Shuffle
- E.g. *n* total items emitted from *m* mappers
- Reduces network transfer and Disk IO costs
  - In ideal case, `m` items vs. `n` items *written and read from disk*, *transferred over network* (`m<<n`)
- Shuffle, less of an impact
  - If more mapper tasks are present than reducers, higher parallelism for doing groupby and mapper-side partial sort.
  - Local Sort on reducer is based on number of unique keys, which does not change due to combiner.

MapReduce: Recap

- Programmers must specify:
  - **map** (k, v) → <k', v'>*
  - **reduce** (k', v'[]) → <k'', v''>*
- All values with the same key are reduced together
- Optionally, also:
  - **partition** (k', number of partitions) → partition for k'
  - Often a simple hash of the key, e.g., hash(k') mod n
  - Divides up key space for parallel reduce operations
  - **combine** (k', v') → <k', v'>*
  - Mini-reducers that run in memory after the map phase
  - Used as an optimization to reduce network traffic
- The execution framework handles *everything else*…

What Does it Solve?

- Scalable data analytics
- Data not viewed or constrained by disk read/writes but find meaning in data via computation over sets of keys and values.
- Abstract out lots of programming details.
- Forgo removing data redundancy via normalization of tables.
  - The penalty of normalization is reading a record requires reading from multiple sources/tables/files.

“Everything Else”

- The execution framework handles everything else…
  - Scheduling: assigns workers to map and reduce tasks
  - “Data distribution”: moves processes to data
  - Synchronization: gathers, sorts, and shuffles intermediate data
  - Errors and faults: detects worker failures and restarts
- Limited control over data and execution flow
  - All algorithms must be expressed in m, r, c, p
- You don’t know:
  - Where mappers and reducers run
  - When a mapper or reducer begins or finishes
  - Which input a particular mapper is processing
  - Which intermediate key a particular reducer is processing

Map-Reduce

- Failures are handled via the jobtracker and the tasktracker.
- The tasktracker notices failures of tasks including
  - Runtime exceptions
  - Sudden exit of the code executing the task
  - Tasks hanging – observed via timeouts
- If the jobtracker itself fails, Map-Reduce has no mechanisms to save and restore computations.

More Recent Platforms

- Notice that Map-Reduce is already two decades old.
- Clearly, a lot has changed since.
- In the following, we will list a few platforms that borrowed ideas from Map-Reduce and provided further abstractions or applied these to other settings.

Other Variants of Map-Reduce

- YARN – Yet Another Resource Negotiator, 2010 release
- Developed to address the scalability concerns of Map-Reduce once the number of nodes is of the order of 10^4 or more.
- YARN splits the responsibilities of the jobtracker into two separate entities
  - a resource manager to manage the use of resources across the cluster, and
  - an application master to manage the lifecycle of applications running on the cluster.
- YARN has more sophisticated mechanisms to save state of the jobtracker on failure.
- YARN is developed to be more general than MapReduce,
  - In fact MapReduce can be seen as just a type of YARN application

More Recent Platforms

- Support for streaming data
  - Recall that Map-Reduce was designed with processing of batch data as the primary objective.
  - However, of late, data has become more dynamic/ streaming in nature.
  - So, need platforms that can work with such streaming data.
  - Platforms such as Kafka support distributed computing on streaming data.
  - FlumeJava is a library developed by Google for building, composing, and running data-parallel pipelines in a declarative and efficient way.
    - Apache Beam and Google Dataflow generalize the ideas from Flume Java and support both batch and stream processing.

More Recent Platforms

- There exist platforms that focus on specific domains such as
  - machine learning (Tensorflow, Pytorch, Mahout)
  - linear algebra (Mahout)
  - and so on.

- Platforms providing higher abstractions:
  - Mahout is designed to build scalable, distributed machine learning algorithms on top of big data platforms such as Hadoop, Apache Spark, and Flink.

Further Reading and Action Items

- Several piece-meal tutorials available online.
- A one-place material available as online book, Hadoop: The Definitive Guide, by Tom White.
  - Will post this PDF in our course moodle.
  - Read chapters 1, 2, 3, and 6.

Other Distributed Programming Frameworks

- There have been distributed programming frameworks both prior and latter to MPI.
- Examples:
  - Remote Procedure Call
  - CORBA
  - RMI – Remote Method Invocation
  - gRPC
  - Erlang
  - More low level mechanisms also exist
    - Sockets
- Some similarities and some differences exist between these frameworks.

Other Distributed Programming Frameworks

- CORBA and RPC are not very popular today.
- RMI and sockets are too low level
- gRPC is widely used – we will detail it briefly now.

A Brief Introduction to gRPC

- An extension of Remote Procedure Call (RPC) first introduced by SUN.
- RPC essentially provides an abstraction for allowing (client) programs to make function calls that get executed on a remote machine (server).
- The abstraction is transparent for the user to issue the function call.
- RPC internally uses protocols such as UDP or TCP to
  - prepare the function call,
  - contact the server for executing the function, and
  - report the results back to the client program.

A Brief Introduction to gRPC

- Implementations of RPC, such as Java RMI, CORBA, added an object-oriented interface to RPC.
- Google introduced gRPC in 2015 based on its own in-house RPC framework, Stubby.
- Stubby allowed multiple microservices within the Google datacenters to connect and communicate with each other.
- gRPC is an open-source framework that is general-purpose and cross-platform ready.

Using gRPC

- To implement the server side functionality, we first define a few functions in the Interface Definition Language (IDL).
- An example simple program to greet a client is as follows.
- Three steps:
  - Interface code
  - Client code
  - Server code

Proto code

```
syntax = "proto3";
option java_multiple_files = true;
option java_package = "com.example.grpc";
option java_outer_classname = "HelloProto";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string name = 1;  //defines the response object structure that the gRPC
                    //server will send back to the client.
}

message HelloReply {
  string message = 1;
}
```

The gRPC Server

```
public class HelloServer {

    public static void main(String[] args) throws IOException,
InterruptedException {
        // Create gRPC server listening on port 50051
        Server server = ServerBuilder
                .forPort(50051)
                .addService(new GreeterImpl())
                .build();

        System.out.println("Starting server on port 50051...");
        server.start();

        System.out.println("Server started.");
        server.awaitTermination(); // keep the server running
    }
```

The gRPC Server

```
static class GreeterImpl extends GreeterGrpc.GreeterImplBase {
        @Override
        public void sayHello(HelloRequest request,
                             StreamObserver<HelloReply> responseObserver) {
            // Build response
            HelloReply reply = HelloReply.newBuilder()
                    .setMessage("Hello, " + request.getName() + "!")
                    .build();

            // Send response
            responseObserver.onNext(reply);
            responseObserver.onCompleted();
        }
    }
```

The gRPC Client

```
public class HelloClient {
    public static void main(String[] args) {
        // Create a channel to the server
        ManagedChannel channel =
    ManagedChannelBuilder
                .forAddress("localhost", 50051) //
    server address and port
                .usePlaintext() // disable TLS for
    local testing
                .build();
        // Create a stub (blocking style)
        GreeterGrpc.GreeterBlockingStub stub =
    GreeterGrpc.newBlockingStub(channel);


//contd -----
```

```
// Build the request
HelloRequest request =
    HelloRequest.newBuilder()
            .setName(“NAME")
            .build();


// Call the service
        HelloReply response =
    stub.sayHello(request);
        System.out.println("Server replied: "
    + response.getMessage());


        // Shutdown channel
        channel.shutdown();
    }
}
```

Other Features of gRPC

- gRPC allows for streaming support.
- Streaming allows for the server (client) to send multiple replies (requests) to one request from (to) a client (server).
- These responses need not all be sent at once and can be sent as the server obtains more data that matches the response.
- The server marks the end of the stream by sending the status details of the server and trailing metadata to the client.

Other Features of gRPC

- Authentication via Transport Layer Security (TLS) is another feature of gRPC.
- Interceptors is another nice feature of gRPC
  - Interceptors are pieces of code that get invoked prior to the RPC call.
  - These act as useful mechanisms to perform any pre- or post-processing as part of the RPC call.
  - Some functionality that one can push to such interceptor routines include logging, metrics, authentication, and the like.
- Deadlocks, timeouts, cancellations, and error handling, are other additional features supported.

Next Steps

- Homework 2 is to be assigned to teams of 2 students each.
- TA will post the details of the team, account for each team, and the problem set for each team shortly.
- Team rules
  - Teams are assigned randomly. Students on audit are excluded.
  - It is the responsibility of the team to work together.
  - If one of the teammates slacks off and agrees so, the other teammate will get evaluated on the work done and prorated, while the laggard will get the same score but with a negative sign.